Discovery Executive Summary

Project: discovery-14-aug · **Generated:** 14/08/2026, 12:05:34

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Performance & Sustainability Analysis	—

---

1. Performance & Sustainability Analysis

EXECUTIVE SUMMARY

The Klearcom platform has significant runtime-performance deficiencies driven by quadratic-complexity tree-building algorithms (six sites across PHP and JavaScript), unbounded in-memory data growth (five high-memory sites including uncapped SSE event fetches and an ever-growing in-memory store), and infrastructure without resource constraints or autoscaling. The SSE streaming architecture uses inefficient polling that re-fetches all session events every 500ms instead of using change streams or cursor-based pagination, compounding both memory pressure and network overhead. Docker containers run with no CPU/memory limits, and the CI pipeline lacks any dependency or layer caching — every push compiles the MongoDB PHP extension from source and downloads all packages. The sustainability posture is partial: always-on containers with no right-sizing, energy-inefficient polling patterns, and no carbon-aware scheduling. The most urgent risks are algorithm efficiency (P1), memory utilization (P4), resource provisioning (P8), and build efficiency (P10), all rated High Risk.

7.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	<span class="rating rating-good">Good	<span class="rating rating-moderate">Moderate	<span class="rating rating-high-risk">High Risk	Measured	Rating
P1	Algorithm Efficiency	High-complexity algorithm sites	0	1–5	>5	6	<span class="rating rating-high-risk">High Risk
P2	Database Performance	Deferred → Backend Modernization (H14/H10)	—	—	—	See Backend Modernization	— (deferred)
P3	API Performance	Response-latency hotspots	0	1–5	>5	3	<span class="rating rating-moderate">Moderate
P4	Memory Efficiency	High-memory sites	0	1–3	>3	5	<span class="rating rating-high-risk">High Risk
P5	CPU Efficiency	CPU-intensive operations	0	1–5	>5	2	<span class="rating rating-moderate">Moderate
P6	Concurrency	Parallelizable work + pool sizing (blocking-I/O → Backend Modernization H14)	0	1–5	>5	3	<span class="rating rating-moderate">Moderate
P7	Caching	Deferred → Backend Modernization H14 / Frontend Modernization H11	—	—	—	See those reports	— (deferred)
P8	Resource Utilization	Over-provisioned / idle resources	0	1–3	>3	4	<span class="rating rating-high-risk">High Risk
P9	Network Efficiency	Excessive-traffic sites	0	1–5	>5	5	<span class="rating rating-moderate">Moderate
P10	Build Efficiency	Build/test pipeline efficiency	efficient	partial	slow / no caching	no caching	<span class="rating rating-high-risk">High Risk
P11	Logging Efficiency	Excessive-logging sites	0	1–10	>10	0	<span class="rating rating-good">Good
P12	Sustainability	Resource-optimization posture	optimized	partial	wasteful	partial	<span class="rating rating-moderate">Moderate

7.5 Actions Required

Hotspot	Action	Rating	Priority
P1 Algorithm Efficiency	Replace recursive `buildTree()` with hash-map grouping; deduplicate copies; derive subsets from existing data	<span class="rating rating-high-risk">High Risk	<span class="sev sev-critical">Critical
P4 Memory Efficiency	Add limits to `getTestEvents()`; cap in-memory store growth; use cursor-based SSE fetch	<span class="rating rating-high-risk">High Risk	<span class="sev sev-critical">Critical
P8 Resource Utilization	Add Docker resource limits; serve production frontend build; introduce autoscaling	<span class="rating rating-high-risk">High Risk	<span class="sev sev-critical">Critical
P10 Build Efficiency	Add `actions/cache` for Composer, npm, and PECL extensions; move `composer install` into Dockerfile layer	<span class="rating rating-high-risk">High Risk	<span class="sev sev-critical">Critical
P3 API Performance	Replace SSE polling with change streams; add pagination to unbounded endpoints	<span class="rating rating-moderate">Moderate	<span class="sev sev-high">High
P9 Network Efficiency	Enable gzip in nginx; set CORS max_age; reduce redundant polling during SSE	<span class="rating rating-moderate">Moderate	<span class="sev sev-high">High
P5 CPU Efficiency	Move test simulation to queue workers; cache serialized MongoDB documents	<span class="rating rating-moderate">Moderate	<span class="sev sev-medium">Medium
P6 Concurrency	Configure PHP-FPM pool sizing; add MongoDB connection pool options; use SQL aggregation	<span class="rating rating-moderate">Moderate	<span class="sev sev-medium">Medium
P12 Sustainability	Replace polling with event-driven architecture; add resource limits; cache CI dependencies	<span class="rating rating-moderate">Moderate	<span class="sev sev-medium">Medium

7.6 Expected Outcomes

- Replacing $O(n^2)$ `buildTree()` with hash-map grouping cuts tree-rendering latency from $O(n^2)$ to $O(n)$, preventing latency spikes as IVR trees grow beyond demo scale.
- Adding limits to `getTestEvents()` and using cursor-based incremental fetch reduces per-SSE-poll memory allocation by 80-95%, eliminating the risk of PHP-FPM OOM kills during concurrent tests.
- Docker resource limits prevent runaway containers from starving sibling services, and production frontend serving eliminates the overhead of Vite's dev-mode file-watching and HMR.
- CI dependency caching (Composer, npm, PECL MongoDB extension) is expected to reduce build times by 60-90 seconds per run, saving ~30+ minutes of CI compute daily for an active team.
- Enabling gzip compression and CORS preflight caching reduces API response sizes by 70-80% and eliminates redundant OPTIONS requests, lowering bandwidth cost and improving perceived latency.","stop_reason":"end_turn","session_id":"3eb72a7e-7b74-4553-bbdd-7acff735a377","total_cost_usd":3.1195115,"usage":
{"input_tokens":4408,"cache_creation_input_tokens":93533,"cache_read_input_tokens":739681,"output_tokens":33097,"server_tool_use":
{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":
{"ephemeral_1h_input_tokens":93533,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":
[{"input_tokens":1,"output_tokens":1873,"cache_read_input_tokens":104188,"cache_creation_input_tokens":11169,"cache_creation":
{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":11169},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":
{"inputTokens":8741,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.008821,"contextWindow":200000,"maxOutputTokens":32000
opus-4-6":
{"inputTokens":4408,"outputTokens":33097,"cacheReadInputTokens":739681,"cacheCreationInputTokens":93533,"webSearchRequests":0,"costUSD":2.1546355,"contextWindow":200000,"maxOutpu
sonnet-4-6":
{"inputTokens":12,"outputTokens":39133,"cacheReadInputTokens":41730,"cacheCreationInputTokens":95068,"webSearchRequests":0,"costUSD":0.9560549999999999,"contextWindow":200000,"ma
[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"78c82c4b-fbdb-49d4-8561-f5d7f187f47f"}